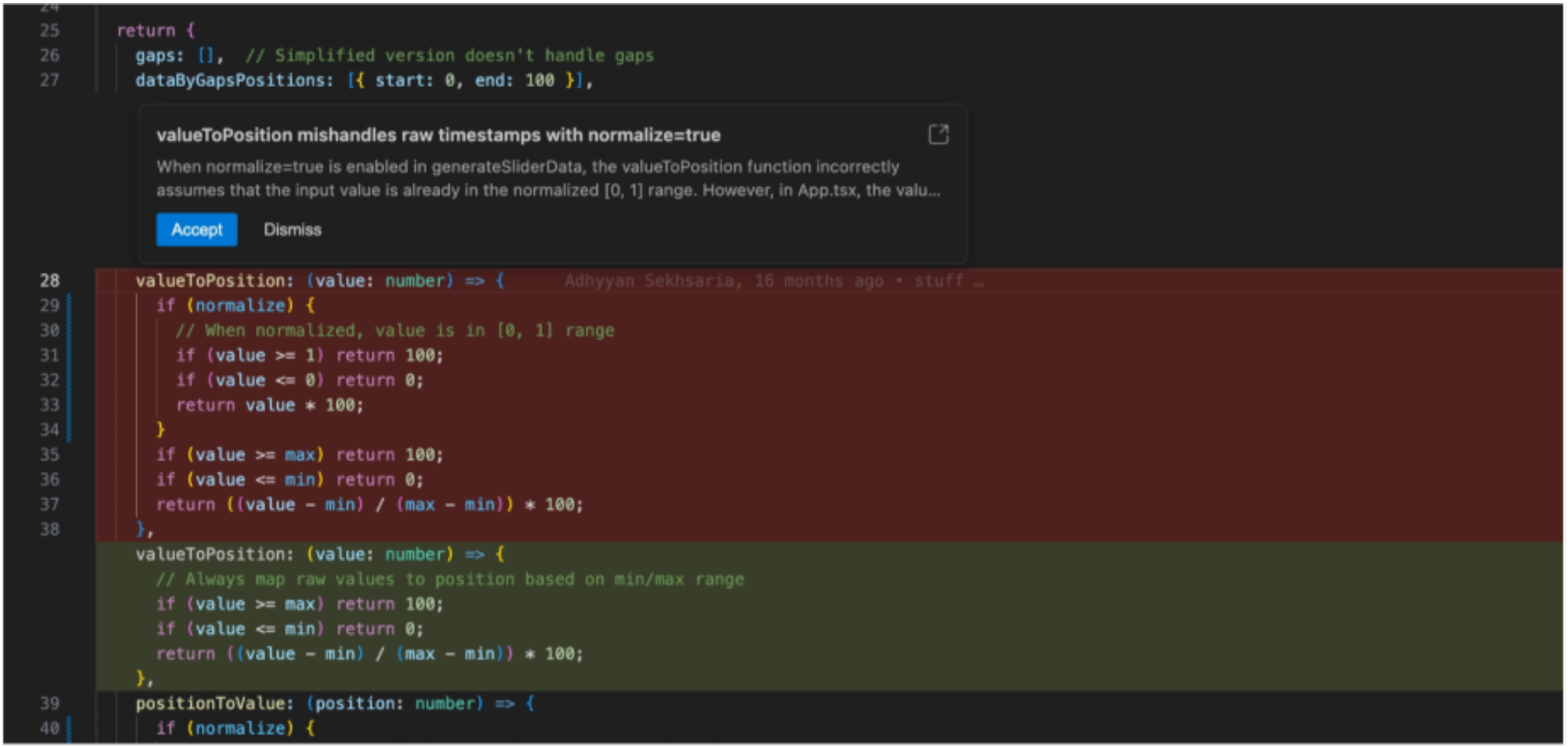
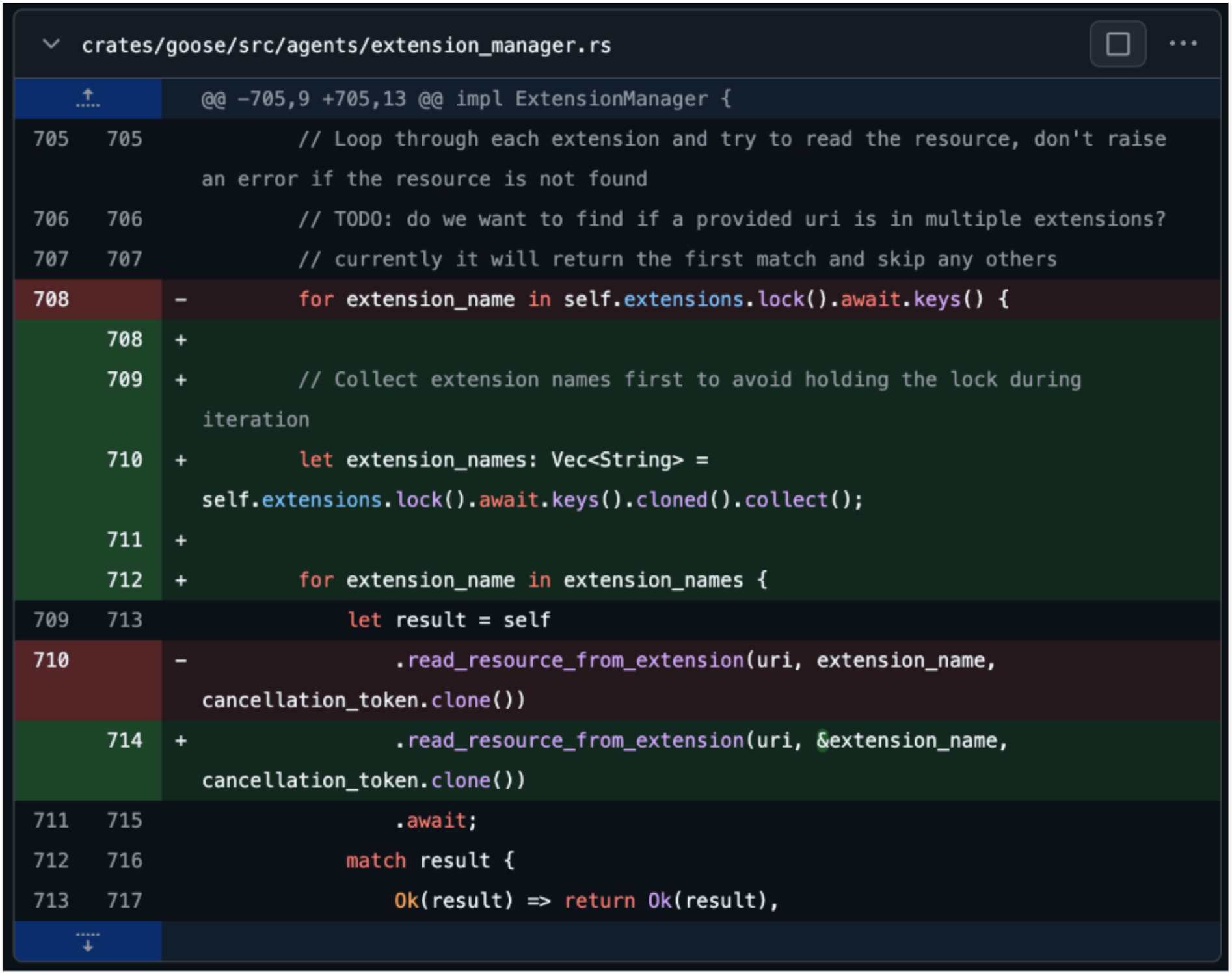

return {
gaps: [], // Simplified version doesn't handle gaps
dataByGapsPositions: [{ start: 0, end: 100 }],
valueToPosition mishandles raw timestamps with normalize=true
When normalize=true is enabled in generateSliderData, the valueToPosition function incorrectly assumes that the input value is already in the normalized [0, 1] range. However, in App.tsx, the valu...
Accept
Dismiss
valueToPosition: (value: number) => {
Adhyyan Sekhsaria, 16 months ago • stuff …
if (normalize) {
// When normalized, value is in [0, 1] range
if (value >= 1) return 100;
if (value <= 0) return 0;
return value * 100;
}
if (value >= max) return 100;
if (value <= min) return 0;
return ((value - min) / (max - min)) * 100;
},
valueToPosition: (value: number) => {
// Always map raw values to position based on min/max range
if (value >= max) return 100;
if (value <= min) return 0;
return ((value - min) / (max - min)) * 100;
},
positionToValue: (position: number) => {
if (normalize) {


crates/goose/src/agents/extension_manager.rs
@@ -705,9 +705,13 @@ impl ExtensionManager {
// Loop through each extension and try to read the resource, don't raise an error if the resource is not found
// TODO: do we want to find if a provided uri is in multiple extensions?
// currently it will return the first match and skip any others
- for extension_name in self.extensions.lock().await.keys() {
+
+ // Collect extension names first to avoid holding the lock during iteration
+ let extension_names: Vec<String> = self.extensions.lock().await.keys().cloned().collect();
+
+ for extension_name in extension_names {
let result = self
- .read_resource_from_extension(uri, extension_name, cancellation_token.clone())
+ .read_resource_from_extension(uri, &extension_name, cancellation_token.clone())
.await;
match result {
Ok(result) => return Ok(result),

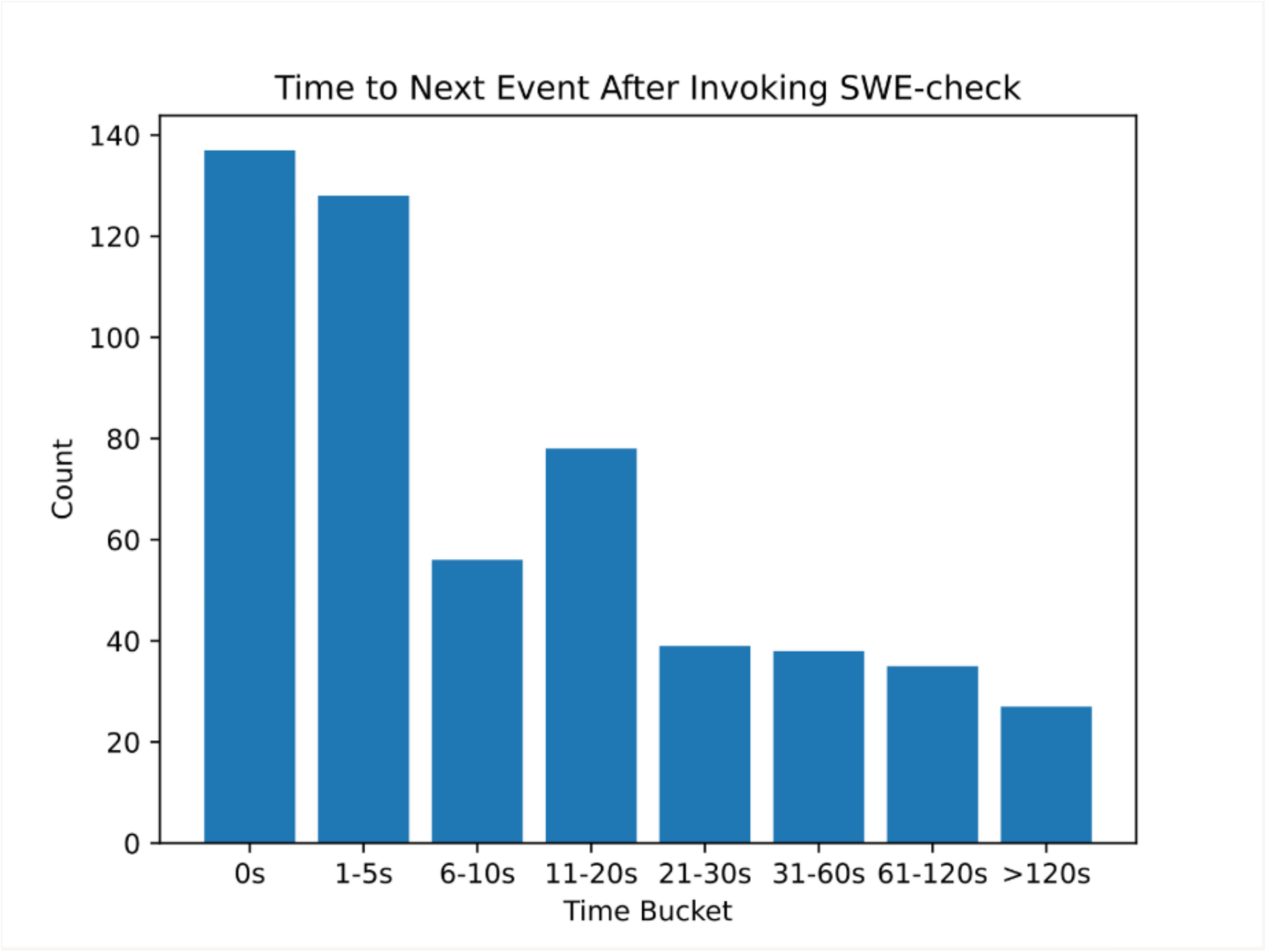
https://cognition.com/blog/swe-check-10x-faster

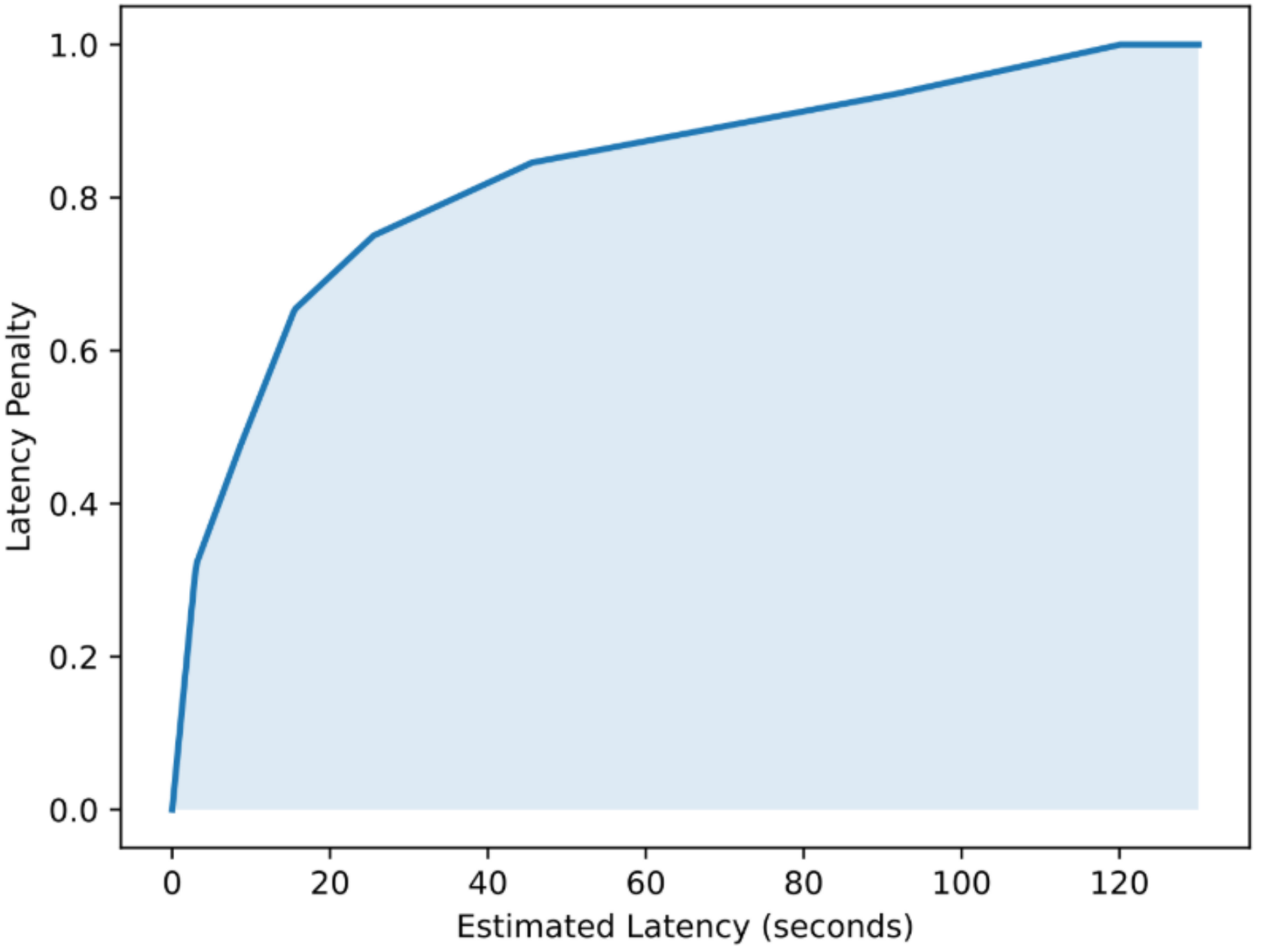
$$f_\beta = \frac{(1+\beta^2)\, P_{\text{pop}} \cdot R_{\text{pop}}}{\beta^2 P_{\text{pop}} + R_{\text{pop}}}$$

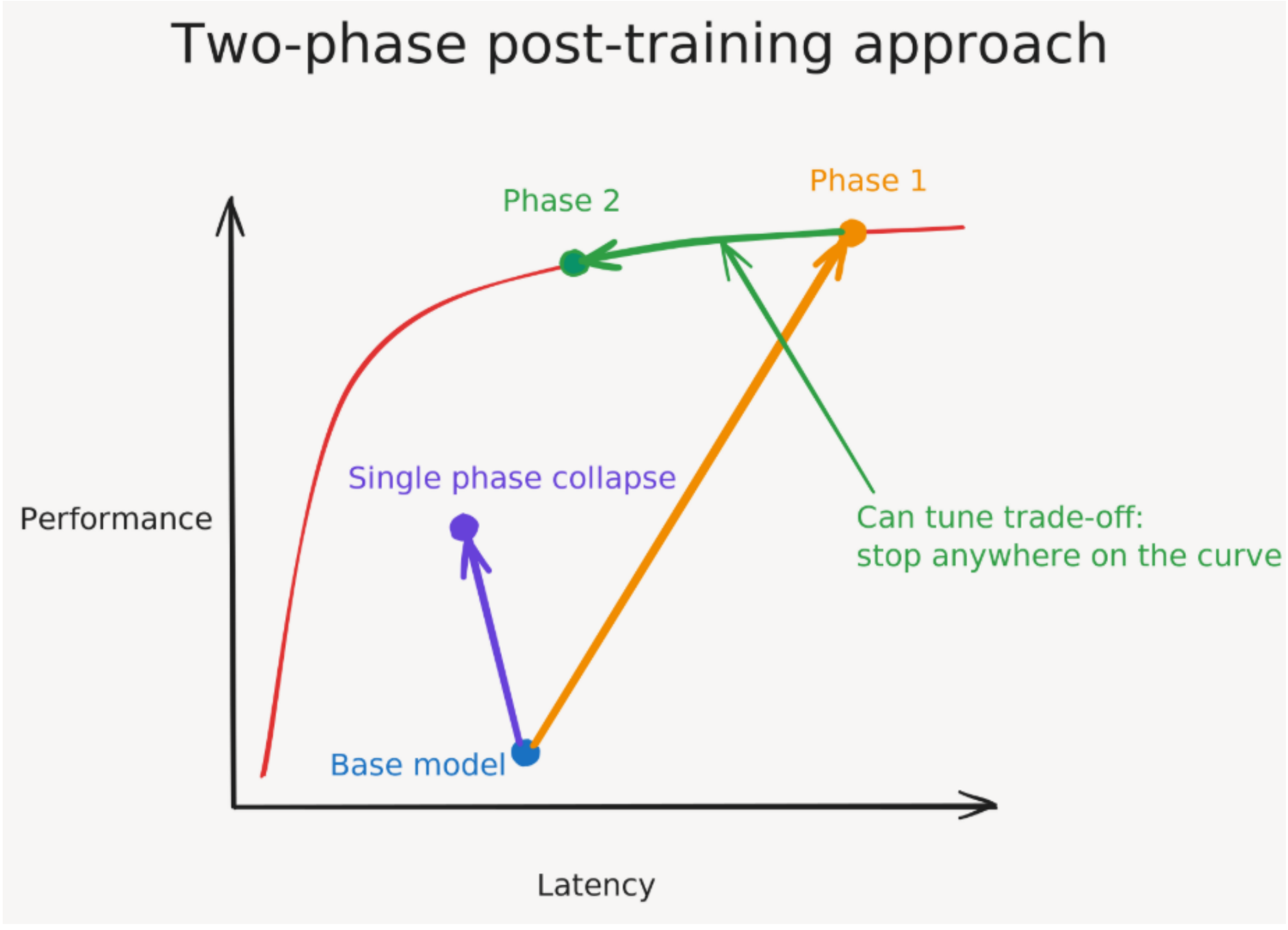
$$f_\beta(\tau) = \frac{(1+\beta^2)\,P(\tau)\,R(\tau)}{\beta^2\,P(\tau) + R(\tau)}$$

$$f_\beta \approx x\,(P_{\text{pop}} - P_{\text{pop, init}}) + y\,(R_{\text{pop}} - R_{\text{pop, init}}) + \frac{(1+\beta^2)\,P_{\text{pop, init}} \cdot R_{\text{pop, init}}}{\beta^2\,P_{\text{pop, init}} + R_{\text{pop, init}}}$$

$$\text{reward}(\tau) = x\left(P(\tau) - P_{\text{pop, init}}\right) + y\left(R(\tau) - R_{\text{pop, init}}\right) + \frac{(1+\beta^2) P_{\text{pop, init}} \cdot R_{\text{pop, init}}}{\beta^2 P_{\text{pop, init}} + R_{\text{pop, init}}}$$


Time to Next Event After Invoking SWE-check
Count
0
20
40
60
80
100
120
140
0s
1-5s
6-10s
11-20s
21-30s
31-60s
61-120s
>120s
Time Bucket


1.0
0.8
0.6
0.4
0.2
0.0
Latency Penalty
0
20
40
60
80
100
120
Estimated Latency (seconds)


Two-phase post-training approach
Phase 1
Phase 2
Single phase collapse
Can tune trade-off:
stop anywhere on the curve
Performance
Base model
Latency